

INSTITUTO FEDERAL DO PIAUÍ

Curso de Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
Programação para Internet II, Prof. Rogério Silva, ADS IV – 2026.2

Atividade C - API REST Profissional: Camadas, Erros, Validação e Upload

O que você já sabe fazer

Desde a rodada 1 do Tópico 2, sua API já:

- responde com os verbos e status codes certos (200, 201, 400, 404, 409);
- está classificada no **Nível 2 do Modelo de Maturidade de Richardson** (recursos + verbos + status semânticos).

O que falta é **organização profissional**: hoje, tudo — roteamento, tradução HTTP e regra de negócio — mora dentro de `src/server.ts`. Um Pleno/Sênior não entrega assim. Esta atividade é sobre transformar o que já funciona em algo que uma equipe consegue manter.

O que você vai entregar ao final

1. O mesmo comportamento de hoje (endpoints de `Patient` e `Encounter`), agora organizado em camadas (Route → Controller → Service).
2. Todo erro da API passando por um único ponto: o `errorHandler`.
3. Validação de entrada com Zod na criação de paciente.
4. Um endpoint novo: upload de foto do paciente, com regras de segurança.

Antes de começar

- Baixe o projeto-base e rode:

```
npm install
```

```
npm run db:reset
```

```
npm run dev
```

- Abra `requests.http` e confirme que os blocos 1–10 respondem exatamente como o comentário de cada um descreve. **Não avance sem isso funcionando.**
- Leia o `roteiro de Refatoração abaixo` ele lista os 14 TODOs na ordem certa. Esta atividade é o passo a passo *de dentro* desses TODOs.

Nível 1 — Essencial

Parte A — Arquitetura em Camadas (Encontro 1)

Siga os TODOs 1 a 7 do projeto-base, nesta ordem:

1. **TODO 1–3 (Patient):** crie `routes/patients.routes.ts`, `controllers/patients.controller.ts` e `services/patients.service.ts`. Migre o código de `server.ts` respeitando a regra:
 - Route só roteia.
 - Controller só traduz HTTP ↔ domínio (lê `req`, chama o Service, formata `res`).
 - Service só decide (SQL, regra de negócio, tradução `snake_case` → `camelCase`).
2. **TODO 4–6 (Encounter):** repita o mesmo padrão para `Encounter`. Atenção ao `Router({ mergeParams: true })` — sem isso, `req.params.id` não chega ao controller de encounters.
3. **TODO 7:** troque as rotas antigas em `server.ts` pelos novos routers.

Critério de "pronto" desta parte: rode `requests.http` (blocos 1–10) de novo, do zero, sem alterar nenhuma requisição. Se qualquer status code ou corpo de resposta mudou, a refatoração introduziu um bug — não avance até corrigir.

Antes de adicionar qualquer camada extra que não está nos TODOs: pare e pergunte — isso resolve um problema que o projeto tem *agora*, ou é só "parecer mais profissional"? Camada em excesso, principalmente quando gerada por IA, tende a fragmentar o código sem necessidade. Veja o Guia de Bolso de IA se tiver dúvida.

Parte B — Tratamento de Erros (Encontro 2)

4. **TODO 8:** implemente a hierarquia em `errors/HttpError.ts`:
 - `HttpError` (base) com `statusCode`, `message`, `details?`
 - `BadRequestError` → 400
 - `NotFoundError` → 404
 - `ConflictError` → 409
 - `UnprocessableEntityError` → 422
 - `PayloadTooLargeError` → 413
5. **TODO 9:** implemente `middlewares/errorHandler.ts`. Todo erro deve sair no formato:

```
{ "error": { "message": "...", "statusCode": 404, "details": null } }
```

Registre-o em `server.ts` com `app.use(errorHandler)` — **por último**, depois de todas as rotas.

- Volte aos Services de Patient e Encounter e troque qualquer resposta de erro manual por `throw new AlguemHttpError(...)`.

Critério de "pronto" desta parte: nenhum `res.status(...).json({ error: ... })` sobrevive fora de `errorHandler.ts`. Rode `requests.http` de novo — os blocos de falha proposital (6, 7, 10) continuam com o status certo, agora no formato novo.

Parte C — Validação com Zod (Encontro 2)

- TODO 10:** escreva `validation/patients.schemas.ts` com `createPatientSchema` (campos `name`, `birthDate`, `nationalId`).
- TODO 11:** implemente o middleware genérico `validate(schema)` em `middlewares/validate.ts`. Em caso de falha, ele deve lançar `BadRequestError` com `details` contendo os campos inválidos.
- Monte o middleware na rota de criação: `patientsRouter.post("/", validate(createPatientSchema), patientsController.create)`.

Critério de "pronto" desta parte: descomente o bloco 11 do `requests.http` — um payload com `name` vazio e `birthDate` inválido deve voltar `400` com `details` listando os dois campos.

Nível 2 — Proficiente

Upload de foto do paciente

- TODO 12:** configure `middlewares/upload.ts` com `multer`:
 - `diskStorage` salvando em `uploads/`, com **nome de arquivo gerado pelo servidor** (nunca o nome original enviado pelo cliente — isso é o que evita um ataque de *path traversal*);
 - `fileFilter` aceitando somente `image/jpeg` e `image/png`;
 - limite de `2MB`.
- TODO 13:** implemente o endpoint `POST /api/patients/:id/photo`:
 - `404` se o paciente não existir;
 - `422` se não vier arquivo nenhum;
 - `413` se o multer rejeitar por tamanho (o próprio multer já cuida disso — verifique que o erro chega formatado pelo `errorHandler`);
 - sucesso: `200` com o paciente atualizado, incluindo `photoUrl`.

Critério de "pronto" desta parte: descomente o bloco 12 do `requests.http`, aponte para uma imagem `.jpg` local, e confirme os 4 cenários (sucesso, paciente inexistente, sem arquivo, arquivo grande demais).

Teste extra recomendado: renomeie um arquivo `.txt` para `.jpg` e tente enviar. O filtro por `mimetype` (não só por extensão) deveria rejeitar.

Frontend reativo adaptado

12. TODO 14:

- Em `public/js/errors.js`, implemente `renderApiError(body)` lendo o contrato `{ error: { message, details } }`.
- Em `public/js/api.js`, implemente `uploadPatientPhoto(patientId, file)` usando `FormData` (sem definir `Content-Type` manualmente).
- Adicione um campo de arquivo na tela de paciente com preview local (`URL.createObjectURL`) antes de qualquer envio ao servidor.
- Troque o tratamento de erro manual em `app.js` (no cadastro de paciente) por `renderApiError`.

Critério de "pronto" desta parte: cadastre um paciente com CNS duplicado pela tela (não pelo `requests.http`) e confirme que a mensagem de erro aparece formatada, sem `alert()`.

Nível 3 — Avançado (portfólio)

Escolha **um** dos dois:

- **Justificativa arquitetural por escrito:** no `README.md` do projeto, escreva uma seção curta (5–10 linhas) explicando uma decisão real que você tomou sobre onde traçar a fronteira entre Controller e Service — com um exemplo concreto do seu código.
 - **Auditoria de segurança do próprio upload:** escreva, também no `README.md`, uma checklist de 3 a 5 itens listando o que você verificou manualmente sobre o endpoint de upload (tipo de arquivo, tamanho, nome gerado) e o que você sabe que **ainda não** está protegido (dica: pense em quem pode chamar esse endpoint hoje).
-

Sobre o uso de IA nesta atividade

O uso de IA é permitido e esperado — mas segue o protocolo da disciplina:

- Use o **C-P-T-R-F-A** para estruturar seus prompts.

- Valide toda saída da IA com o **V.E.R.M.** antes de aceitar.
- **Vibe coding e modos autônomos de agente não são permitidos nesta atividade.** Você precisa entender e conseguir explicar cada linha que entrar no seu projeto.
- Ao revisar uma sugestão de camada, arquivo ou abstração extra que a IA propuser, pergunte: *isso resolve um problema que o projeto tem agora, ou é a IA copiando um padrão de tutorial sem necessidade?*

Checklist de entrega

- ☐ `npm run check` (tsc --noEmit) sem erros
- ☐ `requests.http` rodado do início ao fim, blocos 1–13, com os resultados esperados
- ☐ Nenhum `res.status().json({ error })` manual fora de `errorHandler.ts`
- ☐ Upload de foto testado com pelo menos 1 caso de sucesso e 2 de falha
- ☐ Nível 3 (se optar por fazer): seção correspondente no `README.md`